

Table of Contents

1.	Introduction.....	3
1.1.	Scope of the Project.....	3
1.2.	Objectives.....	4
1.3.	Research questions	5
2.	Literature Review	6
2.1.	Traditional Methods	7
2.2.	Machine Learning and Deep Learning	7
2.3.	Long Short Term Memory (LSTM)	8
2.4.	Convolutional Neural Networks (CNN).....	8
2.5.	Multi-Layer Perceptron (MLP)	9
3.	Methodology.....	10
3.1.	Data Collection.....	10
3.2.	Data Preprocessing	11
3.2.1.	Merge bid/ask into mid-prices.....	11
3.2.2.	Remove duplicates and missing values	12
3.2.3.	Normalize data using Standard Scaler.....	12
3.3.	Feature Engineering.....	13
3.3.1.	Time Features	13
3.4.	Technical Indicators	14
3.5.	Market Regime	16
3.6.	Labelling Strategy	16
3.7.	Sequence Creation.....	17
3.8.	Model Architectures	17
3.8.1.	Convolutional Neural Network	17
3.8.2.	Long Short-Term Memory	18
3.8.3.	Multi-Layer Perceptron	19
3.8.4.	Ensemble	19
3.8.5.	Random Forest	20
3.9.	Training Strategy	21
3.9.1.	Focal loss to handle class imbalance	21
3.9.2.	Class weights to emphasize minority classes	21
3.9.3.	Early stopping to prevent overfitting.....	22
3.10.	Evaluation Framework	22
3.10.1.	Backtesting.....	22
3.10.2.	Metrics	22
3.11.	Statistical Testing	24
3.12.	Walk-Forward Validation.....	24
4.	Implementation	25

4.1.	Libraries.....	25
4.1.1.	TensorFlow / Keras	25
4.1.2.	Scikit-learn for preprocessing.....	25
4.1.3.	Pandas/NumPy for data handling	25
4.1.4.	Matplot and Seaborn for visualization.....	25
4.2.	System Architecture Overview.....	26
4.2.1.	Data Preparation – prepare_data().....	26
4.2.2.	Model Training – run_cnn(), run_lstm(), run_mlp()	28
4.2.3.	Backtesting Engine – run_backtest().....	28
4.2.4.	Performance Evaluation – compute_metrics()	29
4.2.5.	Validation Framework – walk_forward_proper()	30
5.	Testing and Evaluation	31
5.1.	Model Performance	31
5.1.1.	Prediction accuracy	31
5.1.2.	Financial returns	31
5.2.	Benchmark Comparison	31
5.2.1.	Buy & Hold	31
5.2.2.	Moving Average strategy	32
5.2.3.	Transaction Cost Analysis.....	32
5.3.	Statistical Testing	32
5.4.	Ablation Study.....	33
5.5.	Error Analysis.....	33
6.	Discussion.....	35
6.1.	Interpretation of Model Performance	35
6.1.1.	Overall Model Performance	35
6.1.2.	Instrument-Specific Behaviour.....	38
6.1.3.	Performance of Individual Models	39
6.2.	Financial Performance vs Prediction Accuracy.....	41
6.3.	Benchmark Comparison	42
6.4.	Impact of Transaction Costs	42
6.5.	Statistical Significance of Results	43
6.6.	Ablation Study and Feature Importance.....	43
6.7.	Error Analysis Across Market Regimes	44
6.8.	Walk-Forward Validation and Generalisation.....	44
7.	Conclusion	46
7.1.	Limitations of the Study	46
7.2.	Implications and Future Work.....	46

1. Introduction

The financial market over a long period of time has been one of the highly traded markets in the world and from history this started happening as early as the 1600. The financial market is made of several key factors. Which makes it what it is today. One of the key components of the financial market is the instruments, which is what you use to make your trades, and it is made of several instruments such as Stocks, Foreign exchange, precious metals, ETFs, Cryptocurrencies extra. And for one to be able to be profitable in the financial market, the individual must be able to predict these various financial instruments, volatility that is whether the price of a particular instrument will go up or down. And for one to be able to make these predictions there are certain information that an individual or a firm must possess to be able to adequately predict these instruments and be profitable. Some of this information range from financial news, indicators, trends, visuals (charts). Using charts to predict the financial market has been one of the easiest ways to analyse the financial market without considering the cumbersome tasks of using mathematical and rigorous processes.

With the advent of Artificial Intelligence, Machine Learning and Deep Learning some of these information or data (past or present) has been used to be able to adequately predict the market and how these predictions can aid an individual / firm to be profitable in the market.

However, it remains a very challenging task to be able to be able to accurately predict the financial market since the financial market is one that that is of a time series and the financial market is not stagnant and keeps changing due to different factors.

And for this project, it aims to adequately analyse the market through the use of Japanese candlestick patterns and use these patterns to predict whether the market will go up, down or hold using different engineering features.

1.1. Scope of the Project

This project focuses on the following key attributes which are: designing, implementing, and evaluating a machine learning-based trading system using historical financial data.

Included in the project are several things such as the following highlighted below and not limited to

- A multi-asset analysis from four major instruments (forex, commodities, crypto, indices)
- A Feature engineering from Open High Low Close (OHLC) data gotten from Dukascopy demo trading platform
- A Supervised classification of candlesticks to determine three classes (BUY / HOLD / SELL)

- The development of Deep Learning Models (CNN, LSTM, MLP)
- An Ensemble modelling which involves the combination of models
- It consists of a Walk-forward validation
- A Back-testing procedure with transaction costs
- Statistical significance testing
- Benchmark comparison (Buy & Hold, Moving Average)

Excluded:

- The deployment of live trading
- An infrastructure that consists of high frequency trading
- The existence of Fundamental data (macroeconomic indicators)

1.2. Objectives

The primary objective of this project is for machine learning models to use Japanese candlesticks patterns to adequately predict the market direction which is up/down/hold. This major objective has been met through several micro-objectives listed below

1. **Develop a predictive model for financial time-series classification:** It develops a machine learning system using financial market data (time-series) that adequately classifies Japanese candlestick patterns and analyses past events (indicators, prices, etc) to be able to predict the status of the financial market
2. **Engineer meaningful features from raw price data:** It also uses various features (characteristics) in the financial market as inputs to help identify patterns in the historic dataset in predicting the upward, downward or stagnant trajectory of the financial market
3. **Evaluate multiple deep learning architectures:** This work adequately evaluates and analyses several deep learning models and be able to ascertain a particular model(s) that is/are good enough to adequately predict the financial market using historical time series data.
4. **Implement walk-forward validation to simulate real trading conditions.** This pipeline also evaluates historic time series data and tests it using iterative strategy against real-world trading scenario to ascertain its predictive and profitable tendencies.

5. **Measure financial performance of the Models.** This pipeline also measures the financial performance of the different models together with the different financial market instruments using different performance metrics such as sharp ratio, draw-down and win rate. Which provides comprehensive details and overview of the strategies, efficiency and consistency.
6. **Compare Machine and Deep Learning models to traditional benchmarks.** This work also compares traditional ways of predicting the financial market to that of the ML/DL models. This also shows the shift and the benchmarks of task specific systems to that of machine based one.
7. **Assess statistical significance of trading strategies.** This pipeline analyses, assesses and evaluates the statistical importance of trading strategies whether the profitability of market has been determined by market edge or barely by chance, luck or simple overfitting/underfitting.
8. **Analyse model errors and feature importance.** This pipeline will be critically analysing model errors and feature importance to ascertain and understand the model, behaviours and actions, performance and reliability. It also assesses the where the model succeeds or fail and also the features that drove the model to such market conditions

1.3. Research questions

These are questions that help to shape and guide this research. It steers the research in the right direction and keep it in course to be able to stay within the objectives of the research these questions are highlighted below

1. Does deep learning models outperform traditional trading strategies?
2. Which architecture (CNN, LSTM, MLP) performs best for time-series prediction?
3. Does feature engineering significantly improve model performance?
4. Are model profits statistically significant?
5. How sensitive are strategies to transaction costs?
6. Does walk-forward validation reduce overfitting?
7. Which features contribute most to predictive power?

2. Literature Review

Time Series and financial data is something that has been around since the 1800s. According to Meriam Webster's dictionary, time series is a term that was first used in 1896 and was defined as "a set of data collected sequentially usually at fixed intervals of time". Wikipedia, which is another reputable source, wrote about the first person to study Time Series who was W.S. Jevons. In his book "Investigations in currency and finance" published by MacMillan in 1884. He studied financial time series of bankruptcy that happened between the period of 1825 and 1860. Although he did not provide any textbook definition of time series but was among the forerunners of the word by analysing economical and financial statistics over a period. In the 1976 Wayne A. Fuller in his book Introduction to statistical Time Series also gave similar definition as Merriam Websters own but was more detailed, he stated that Time Series is defined as "a sequence of observations (or data points) ordered in time, typically taken at consistent, equally spaced intervals" Another definition that describes time series is the one from Konar and Bhattacharyan in their book titled **time series prediction and applications** states that "time-series describes a discrete sequence of amplitudes of an entity ordered over time" It further explains that a time-series describes the temporal variations in atmospheric temperature, rainfall, humidity, stock price, and any other measurable quantity that has a variation with time and available as a discrete data points sampled uniformly or non-uniformly over time.

However, time series data over the years has been used in several fields such as finance and economics(Forecasting stock prices and market volatility analysis), marketing and retail(sales forecasting, inventory management, analysing customer demand and supply trends), manufacturing and IoT(Predictive maintenance, IoT sensors analysis and detection anomaly), Healthcare and Medicine(Monitoring patient vitals, disease and epidemiology prediction), Environment and Earth Sciences(weather forecasting and seismic monitoring, climate change tracking), Engineering(signal processing and automation control systems) and Business Management(forecasting demand to optimize demand and supply chain). Due to the fact that time series data can be used in finance as stated above is what gave rise to the term financial time series data which according to Yifan Hu paper titled "FinTSB: A Comprehensive and Practical Benchmark for Financial Time Series Forecasting" is defined as the Financial time series refers to a sequence of data points ordered chronologically, typically representing asset price factors or market indicators, that reflect the dynamic behaviour of financial markets. For this study we will focus on financial time series data. As stated above financial time series data can be used to analyse and forecast instruments in the financial market.

2.1. Traditional Methods

The first traditional financial time series analysis was done in the **1900** by French mathematician Louis Bachelier, in his doctoral thesis *Théorie de la spéculation*, model financial returns as an uncorrelated random walk, laying the foundation for modern quantitative finance and a theory known as the Brownian Theory. Whilst analysing financial time series data it is but necessary we mention the traditional models which are the Autoregressive Integrated Moving Average (ARIMA), Generalized Autoregressive Conditional Heteroskedasticity (GARCH), Vector Auto Regression VAR) have been used widely over the years for financial instrument prediction such as stocks, and forex due to their ability to model linear patterns and volatility. Ayoub in his work “An ARIMA approach sub-sector analysis at Amman Stock Exchange” applied ARIMA to analyse sub-sector stock prices at the Amman Stock Exchange, achieving reliable forecasts for linear trends.

Ma'mon Abu Hammad et al in his Comparative Analysis of Bilinear Time Series Models with Time-Varying and Symmetric GARCH Coefficients: Estimation and Simulation states that in financial time series analysis, Generalized Autoregressive conditional heteroskedasticity (GARCH) models have become essential tools which capture time varying variances and volatility clustering. He further asserts that they offer insights into economic forecasting, asset pricing, risk management and option valuation. Similarly, Suripto utilized ARIMA to predict stock price volatility during the COVID-19 pandemic, highlighting its effectiveness in stable market conditions. However, these models often struggle with capturing nonlinear dynamics, limiting their predictive power in volatile markets like the Tehran Stock Exchange (TEPIX)

2.2. Machine Learning and Deep Learning

Machine Learning has been a trending phenomenal over the last decade and according to International Business Machines (IBM) Corporation defined as the “subset of artificial intelligence (AI) focused on algorithms that can “learn” the patterns of training data and, subsequently, make accurate *inferences* about new data. And in financial time series analysis recognizing patterns is an integral part. Machine Learning over the years has been a dominant part in the artificial intelligence field it provides the backbone of modern AI systems from predicting models to Large Language Models. And in Machine Learning and deep learning there are different models and architectures in ensuring it becomes more efficient in processing and predicting large data. Traditional methods such as ARIMA have been successful in predicting and analysing linear and stable financial time series data but when it comes to complex and unilinear data they tend to be outstretched.

According to an article written by Association for Computing Machinery (digital library) states that in recent years, CNN and GRU in deep learning algorithms have attracted much attention because of their unique advantages. CNN can automatically extract local features from data through convolution operation, which makes it perform well in dealing with short-term fluctuation patterns in financial time series data. Xin Yao Qian asserts that Machine Learning, as an emerging research field in recent years, has brought about many incredible improvements in tasks such as regressing and classifying, and it's also promising to exploit the methodology in financial time series predicting. He further asserts that the precision of financial time series between traditional time series models and mainstream machine learning models are vast as the result shows that machine learning as a modern method far surpasses traditional models in precision and accuracy. Some of these assertions has shown that the emergence of machine learning and deep learning in predicting financial time series data cannot be over emphasized.

2.3. Long Short-Term Memory (LSTM)

The LSTM model research and application is one for the architectures that has been used in the financial time series prediction by many authors around the globe. One of these scholars is Chengkun Liu in his article **“financial data prediction based on LSTM”** asserts that the model performance of LSTM model in training set, verification set and test set is relatively stable and has high prediction accuracy. On the training set, the model can fit the data well during the training process. Another scholar who propounded about this model is Carmina Fjellström in the article **Long Short-Term Memory Neural Network for Financial Time Series** “LSTMs have been shown to be especially suited for time series data due to their ability to incorporate past information, while neural network ensembles have been found to reduce variability in results and improve generalization”. However, the LSTM has it own drawback when it comes to predicting and analysing financial time series data which according to Kandadi et al **Drawbacks of LSTM Algorithm-Case Study** states that the intricate architecture, which consists of several gates and memory cells, makes them computationally costly and increases training time and memory consumption. It also states is has trouble handling lengthy sequences and identifying long term dependencies

2.4. Convolutional Neural Networks (CNN)

Convolutional Neural Networks is a model that was initially developed for image classification but over the years scholars have used it in the prediction of financial time series data. According to Lang Ouyang in the article Research on Financial Time series forecasting model based on machine learning states that CNNs are primarily employed to extract local features from data. By leveraging the operations of convolutional layers and pooling layers, CNNs can autonomously

learn the spatial attributes inherent in the data. While CNN has proved to be a reliable model when it comes to predicting financial time series data it also has its drawback Thejas Kiran **Unlocking the Potential of Convolutional Neural Networks (CNNs) in Time Series Forecasting** states that CNNs excel at learning features from data, interpreting these features can be challenging. Understanding why the model makes specific predictions might be less straightforward compared to traditional statistical methods like ARIMA.

2.5. Multi-Layer Perceptron (MLP)

According to conference paper on **A Non-sequentially Trained MLP Model for Financial Time Series Forecasting** states that MLP architecture combined with feature engineering had better performance comparable to more complex neural network models. De Rojas stated that Artificial Neural Networks such as MLP model have shown superior accuracy compared to traditional techniques such as autoregressive integrated moving average (ARIMA) models. However, like all other machine learning and deep learning models it has its drawbacks and disadvantages it suffers from some drawbacks such as getting trapped in local minima, human intervention during the stage of training, and limitations in architecture design as propounded by Aras et al in **Comparing and Combining MLP and NEAT for Time Series Forecasting**

3. Methodology

This study is one that is conducted on different financial instruments, to predict the movement or direction of the market and how this can be used to embark on profitable trade. In doing this there are several processes that were followed and theories adopted to achieve the aim and objectives of this research. This methodology is a data driven one, which develops and design machine learning and deep learning strategies to replicate real world trading scenarios ensuring, accuracy, robustness, efficiency, economic validity and financial importance is maintained. This methodology consists of data collection and preprocessing, feature engineering, model design, implementation, testing and evaluation etc. Below are the iterative steps highlighted below

3.1. Data Collection

For Research of this nature that is data driven, it is but certain that a high level of financial time series must be acquired for it to be successful. For the data collection phase, a historical bid/ask data for multi-assets on different time frames and period was acquired from a demo trading platform known as Dukascopy that is publicly available for such uses. The multi-asset data on different time frames and different periods was used to ensure that the models developed are robust, generalized and adaptive to different markets and conditions. These four multi asset historic data collected for this research are highlighted below:

- **Indices (S&P 500):** For the indices instrument, the S&P 500 data on a timeframe of five (5) minutes was used on a period covering January 2026 – April 2026. This instrument was used because the S&P 500 is among the most traded indices in the world and it represents equity market. This instrument represents a broader economic sentiment and institutional trading activity. It shows macroeconomic indicators and large-scale market movements.
- **Crypto (BCHUSD):** Bitcoin represents the Cryptocurrency market and is also one of the highest traded cryptocurrencies traded that is known for its very high volatility, speculative behaviour and non-traditional trading pattern. For cryptocurrency a trading time of ten (10) minutes is used for this research and the period it covered is from January 2026 to April 2026 and has been traded against the United States Dollars (USD). This is because unlike other instruments cryptocurrency trades continuously without centralized regulations which makes its data challenging and diverse for the training.
- **Commodities (Gold):** In the financial market Gold as a commodity is regarded as a haven for many traders. For the commodity asset, the data for Gold is collected for the period January 2026 to April 2026 on a trading time frame of fifteen (15) minutes and has been traded against the United States Dollar. This is because of its inclusive exposure to macro economies, which are least volatile compared to other financial market instruments.

- **Forex (GBPUSD):** The foreign exchange instrument is the most traded instrument on the financial market and is called Forex for short. The data collected for this instrument covers the same period as the other instruments from January 2026-April and a trading timeframe of thirty (30) minutes and traded against the British Pounds (GBP). This instrument operates twenty-four (24) hours a day, 5 days a week which makes it high frequency for intraday trading, and it allows models to learn from highly liquid and efficient market
- **Bid/Ask:** In the financial market it is basically operated based on buying and selling which is called bid and ask. And instead of just relying on just the buy end which is the bid or just the ask which is the sell the research then calls for collection of both bid and ask for the different instruments for the data to be balanced and to present a real market situation
- **Data Format:** the data is stored in a CSV format and each bid/ask is made up of Open, High, Low Close and timestamps columns.

Fig 1 Picture of the instruments

```
INSTRUMENTS = {
    "IUFSUSUSD_5m": {"ask": f"{BASE_PATH}/IUFSUSUSD_5 Mins_Ask_2026.01.01_2026.04.13.csv",
                    "bid": f"{BASE_PATH}/IUFSUSUSD_5 Mins_Bid_2026.01.01_2026.04.13.csv"}, #s&p 500 bid and ask data
    "BCHUSD_10m": {"ask": f"{BASE_PATH}/BCHUSD_10 Mins_Ask_2026.01.01_2026.04.13.csv",
                  "bid": f"{BASE_PATH}/BCHUSD_10 Mins_Bid_2026.01.01_2026.04.13.csv"}, #bitcoin/USD bid and ask data
    "XAUUSD_15m": {"ask": f"{BASE_PATH}/XAUUSD_15 Mins_Ask_2026.01.01_2026.04.13.csv",
                  "bid": f"{BASE_PATH}/XAUUSD_15 Mins_Bid_2026.01.01_2026.04.13.csv"}, #gold/USD bid and ask data
    "GBPUSD_30m": {"ask": f"{BASE_PATH}/GBPUSD_30 Mins_Ask_2026.01.01_2026.04.13.csv",
                  "bid": f"{BASE_PATH}/GBPUSD_30 Mins_Bid_2026.01.01_2026.04.13.csv"}, #GBP/USD bid and ask data
}
```

In summary, the data collection process is designed to ensure diversity, realism, and robustness. By incorporating bid/ask data across forex, commodities, cryptocurrencies, and indices, the study establishes a strong empirical foundation for developing and evaluating deep learning-based trading strategies.

3.2. Data Preprocessing

Data Preprocessing is the art of getting the data ready for exploration, and for financial time series data is sometimes raw. Which means it is made up of inconsistencies, noise, structural issues and other data irregularities that will affect the model's performance. To avoid these data issues a preprocessing pipeline is used to put the data in the right format, structure, quality and non-bias nature to be explored.

3.2.1. Merge bid/ask into mid-prices

The original dataset for each instrument consists of bid and ask price and to have a single representative price, both data have been merged to have a mid-price which is arrived at by the formula below:

Formula 1 – Mid-price Formula

$$\text{Mid Price} = \frac{\text{Bid Price} + \text{Ask Price}}{2}$$

Having the mid-price helps reduce the noise that comes with bid and ask spread fluctuations while maintaining a real market approximation of both data. And the mid-price has been used as base in deriving other features ensuring consistency.

3.2.2. Remove duplicates and missing values

Removing duplicates and missing values is a data preprocessing step that is done to achieve data integrity. This is done by removing duplicates rows and columns and missing and infinite values. This ensures only valid and reliable data points are used in the model.

Fig 2 Picture of preprocessing code

```
def clean_data(df): #function to clean data (delete duplicates)
    df = df.drop_duplicates()
    df = df.replace([np.inf, -np.inf], np.nan) #replace infinite values with nan
    df = df.dropna()
    return df.reset_index(drop=True) #reset index
```

3.2.3. Normalize data using Standard Scaler

For models not to be negatively affected by financial features which normally operate on different scales, all features need to be normalized by standardizing it and this is done using the standard scaler to help prevent data leakage. This transforms each feature as follows with the formula below:

Formula 2 –Scaling Formula

$$X_{scaled} = \frac{X - \mu}{\sigma}$$

Where:

- μ is the mean of the feature
- σ is the standard deviation

The scaler is fitted only to the training dataset and then later to the testing and validation dataset. This is done for the training data not to have prior knowledge about the future.

The preprocessing processing stage helps to keep the data clean, structured, and consistent for the machine learning to use. By applying mid-price, removing duplicates and missing values, applying scaling techniques preventing data leakage makes the data ready.

3.3. Feature Engineering

Feature Engineering is a very important aspect in the model pipeline as it transforms data into meaningful representations, capturing market dynamics of the dataset. In the research a host of features has been used to show price patterns and behaviours, indicators, trends, volatility and overall market conditions. These features are highlighted below

3.3.1. Time Features

- Financial market shows temporal patterns and these patterns are determined by different trading sessions, market openings and economic events. To capture these cyclical behaviours, hours of the day and days of the week are extracted from timestamps, these features are then encoded

Formula 3 – Encoding Formula

$$\sin = \sin\left(\frac{2\pi x}{T}\right), \quad \cos = \cos\left(\frac{2\pi x}{T}\right)$$

Where T represents the period 24 hours or 7 days

This encoding makes sure that cyclical relationships (e.g., hour 23 and hour 0) are retained, avoiding artificial discontinuities introduced by linear representations.

Candlestick Structure Features

- **Body:** This is the difference between Opening and closing price
- **Range:** The difference between high and low prices
- **Upper wick:** Distance from the higher of open/close to the high price
- **Lower wick:** Distance from the lower of open/close to the low price
- **Body ratio:** Proportion of the candle body relative to the total range
- **Direction:** Sign of price movement (+1 for bullish, -1 for bearish, 0 for neutral)

Formula 4 – Candlestick formula

$$\text{Body} = |C - O|$$

$$\text{Range} = H - L$$

$$\text{Upper Wick} = H - \max(O, C)$$

$$\text{Lower Wick} = \min(O, C) - L$$

$$\text{Body Ratio} = \frac{|C - O|}{H - L + \epsilon}$$

$$\text{Direction} = \begin{cases} 1 & \text{if } C > O \\ -1 & \text{if } C < O \\ 0 & \text{otherwise} \end{cases}$$

Where O = Open Price

C = Close Price

H = High Price

L = Low Price

These features show knowledge about the buying and selling pressure and the indecision in the market.

3.4. Technical Indicators

A number of widely used indicators was implemented to capture trend and momentum of the financial market.

- **Exponential Moving Averages (EMA 12 and EMA 26):**

It is a price trend indicator that tracks price trend by giving more weights to recent price data. It is used to identify short-term and medium-term trends

Formula 5 – EMA Formula

For period n :

$$EMA_t = \alpha \cdot C_t + (1 - \alpha) \cdot EMA_{t-1}$$

Where:

$$\alpha = \frac{2}{n + 1}$$

- **Moving Average Convergence Divergence (MACD):**

It is trend-following momentum indicator that shows the relationship between two exponential moving averages (EMAs) of a security's price. It helps traders identify changes in trend strength, direction, momentum, and potential entry/exit points through line crossovers and histogram shifts.

Formula 6 – MACD Formula

$$MACD = EMA_{12} - EMA_{26}$$

- **Signal Line:**

Smoothed version of MACD for identifying trend reversals

Formula 6 – Signal Formula

$$Signal = EMA_9(MACD)$$

- **Alligator Indicator (Jaw, Teeth, Lips):**

Multiple moving averages used to identify trend phases

Formula 7 – Alligator Formula

$$Jaw = SMA_{13}(C)$$

$$Teeth = SMA_8(C)$$

$$Lips = SMA_5(C)$$

Where SMA is the simple moving average:

$$SMA_n = \frac{1}{n} \sum_{i=0}^{n-1} C_{t-i}$$

- **Stochastic Oscillators (14, 21, 28 periods):**

Measure momentum by comparing closing price relative to recent highs and lows

Formula 8 – Stochastic Formula

For period n :

$$Stoch_n = \frac{C - L_n}{H_n - L_n + \epsilon} \times 100$$

Where:

- L_n = lowest low over last n periods
- H_n = highest high over last n periods

Used for:

- $n = 14, 21, 28$

These indicators are commonly used in quantitative trading and provide structured signals for trend-following and mean-reversion behaviours.

- **Volatility**

In the financial market, volatility is a very important factor in ascertaining the magnitude of risk and price movement. The following indicators are used to assess the volatility of the market.

- Rolling standard deviation

This is calculating the standard deviation, the amount of variation within a particular sliding window such as 10, 20 and 50 periods

Formula 9– Standard Deviation Formula

$$\sigma_n = \sqrt{\frac{1}{n} \sum_{i=0}^{n-1} (r_{t-i} - \bar{r})^2}$$

Used for:

- $n = 10, 20, 50$

- Average True Range

This measures the average price movement range over time

Formula 10– Average True Range Formula

$$ATR_{14} = \frac{1}{14} \sum_{i=0}^{13} (H - L)$$

3.5. Market Regime

A market regime is a distinct, persistent state or "mood" of a financial market, defined by specific patterns in volatility, direction (trend/sideways), and macro conditions. This has been used because it allows the model to use its prediction based on prevailing market direction, improving decision making

Formula 11 – Market Regime Formula

Trend strength:

$$\text{Trend Strength} = EMA_{12} - EMA_{26}$$

Regime classification:

$$\text{Regime} = \begin{cases} 1 & \text{if } EMA_{12} > EMA_{26} \\ -1 & \text{if } EMA_{12} < EMA_{26} \\ 0 & \text{otherwise} \end{cases}$$

3.6. Labelling Strategy

This exhibits the multi-class classification problem where the aim of this study is to predict price movement based on historical data. Each observation has been assigned to one of the three classes

- BUY – Expected Upward trend
- HOLD – No movement
- SELL – Expected downward trend

Labels are based on future returns over a fixed horizon. And the fixed return is calculated with the formula below

Formula 12 – Fixed Return Formula

- C_t = closing price at time t
- C_{t+h} = closing price at future time $t + h$
- h = prediction horizon (set to 6 time steps)

The **future return** is calculated as:

$$r_t^{(h)} = \frac{C_{t+h}}{C_t} - 1$$

3.7. Sequence Creation

The Financial Market and time series data is inherent which means it learns from observations from historic data and based on this observation can make predictions. So, for it to be able to learn from past observation the raw data is changed into sequences through a sliding window approach. For this study each input consists of 128 timesteps that represents historic market condition. The model is then asked to predict six (6) steps ahead. This is done to differentiate between input and target preventing the model to learn noise and be able to learn patterns and maintain integrity. Due to this process and structure, the model is trained to act as a forward-looking decision rather than a reactive classifier. It also helps prevent data leakage. The only downside to it is the fact that it reduces the total number of reusable samples due to the sequence length and forecast sample.

Figure 3 Sequence Code

```
def create_sequences_from_array(data, labels):  
    X, Y = [], [] #X= input sequence Y=target labels  
  
    for i in range(SEQ_LEN, len(data) - 6): #six steps back  
        X.append(data[i-SEQ_LEN:i])  
        Y.append(labels[i+6]) #predict six steps ahead by looping  
  
    return np.array(X), np.array(Y)
```

3.8. Model Architectures

Financial Time Series data is one that is complex **and** very noisy, and for it to be explored adequately in predicting the market, certain machine learning and deep learning architectures need to be implemented and compared. Each architecture has its own advantages and disadvantages and how best it can explore the financial time series data on certain market situations. For this work we examined four model architectures which are highlighted below:

3.8.1. Convolutional Neural Network

Few of the reasons why CNN was designed is for image classification and to capture temporal patterns within sequences. In the context of financial time series this works better with short term

trends, price formation, micro trends and momentum burst. CNN makes use of filters and these filter captures patterns across time irrespective of its exact position in the sequence. This provides translation variance, which is good because similar market conditions can occur at different times. The CNN model consists of pooling layers that reduce dimensionality while retaining important signals. Added to the pooling layers the dropout layers which are included throughout the model to prevent overfitting by randomly disabling neurons during training. The model then uses a global pooling layer to convert the sequence into a fixed-size representation, followed by fully connected (dense) layers that learn higher-level feature interactions. Finally, the output layer uses a SoftMax activation function to produce probability distributions across the target classes (BUY, HOLD, SELL).

CNN are efficient and tends to train faster than recurrent models. Their main limitation as against other models is that it is challenging for them to maintain long term relationships as they primarily focus on short dependencies.

Figure 4 CNN Code

```
def build_cnn(shape):
    return tf.keras.Sequential([
        layers.Input(shape),#defines input shape
        layers.Conv1D(64,3,activation="relu"), #applies 64 filters in 3 timesteps
        layers.MaxPooling1D(), #reduce size of data and keeps the strongest signal
        layers.Dropout(0.2), #drops 20% nuerons during traning to prevent overfitting
        layers.Conv1D(128,3,activation="relu"), # 128 filters applied more complex patterns
        layers.GlobalMaxPooling1D(), #converts sequence into single vector
        layers.Dropout(0.3), #drops 30% nuerons during traning to prevent overfitting
        layers.Dense(128,activation="relu"), #fully connected
        layers.Dropout(0.4),
        layers.Dense(NUM_CLASSES,activation="softmax")
    ])
```

3.8.2. Long Short-Term Memory

The LSTM model was proposed by Hochreiter and Schmidhuber. The LSTM model is almost directly opposite to the CNN Model, unlike the CNN Model the LSTM model handles long-term sequential dependencies very well and often train data for a long amount of time. This is achieved through a memory system that selectively retains or forgets information over a period. In financial markets, long-term dependencies may depict broader trends, regime shifts, or sustained volatility patterns. The LSTM is therefore well-suited for capturing these effects, which are often missed by simpler models. Input shape (timesteps and features) is then taken and build a model starting with an input layer, an LSTM layer that captures temporal dependencies and patterns over a long period, making it suitable for financial time series data. Dropout is used within the LSTM to reduce overfitting by randomly ignoring portions of the data during training. The extracted temporal features are then passed through fully connected (dense) layers to learn higher-level

representations, with additional dropout for regularisation. Finally, the model outputs predictions through a SoftMax layer, producing probabilities for each class (BUY, HOLD, SELL). However, LSTM model is easily prone to overfitting when the dataset is small and noisy, that is why the use of early stopping is adopted for appropriate regularization, and it is computationally expensive.

Figure 4 LSTM Code

```
def build_lstm(shape):
    return tf.keras.Sequential([
        layers.Input(shape),
        layers.LSTM(64,dropout=0.2,recurrent_dropout=0.2),
        layers.Dense(128,activation="relu"),
        layers.Dropout(0.4),
        layers.Dense(NUM_CLASSES,activation="softmax")
    ])
```

3.8.3. Multi-Layer Perceptron

The Multi-Layer Perceptron Model serves as a baseline model. It consists of one input layer, one or several hidden layers and one output layer. It is a supervised learning model in which the input and expected output is given to the network. It does it just by flattening the sequence into a single vector, it then treats the data as a single input and ignores temporal ordering. Because of its ability to establish relationships, the MLP model is very efficient in prediction and remains one of the simplest models when it comes to Machine Learning and Deep Learning models.

The multilayer perceptron is a feedforward type of network in which the inputs feed the network, propagating toward the outputs in a single direction, with the objective of sending the data from the input layer to the output layer and traversing all hidden layers that exist, which are responsible for processing the information and extracting the characteristic features.

Figure 5 MLP Code

```
def build_mlp(shape):
    return tf.keras.Sequential([
        layers.Input(shape),
        layers.Flatten(),
        layers.Dense(256,activation="relu"),
        layers.Dropout(0.4),
        layers.Dense(128,activation="relu"),
        layers.Dropout(0.3),
        layers.Dense(NUM_CLASSES,activation="softmax")
    ])
```

3.8.4. Ensemble

The Ensemble model was used since it captures different aspects of the data. It works by combining the out of different and multiple models to make its final prediction. For example, CNN is good at

capturing short-term patterns, LSTM is good at capturing long term dependencies and MLP captures overall feature relationships. By averaging their errors, the model reduces the individual model's error impact then makes its prediction and produces more stable results.

The Ensemble Model improves forecasting accuracy, reduces volatility, and mitigates overfitting by combining multiple models. They often outperform individual or stand-alone models as they are robust to noise and handle complex non-linear data. This is used in financial applications when model uncertainty can cause great financial risk. This brings some amount of certainty to the pipeline.

3.8.5. Random Forest

Apart from the deep learning architectures a **Random Forest classifier** is introduced as a traditional machine learning benchmark. For this project the Random Forest model is added to the same feature set used by deep learning models. However, unlike CNN and LSTM models, Random Forest does not inherently process sequential data, it has to **flatten data into fixed-length vectors** before being pushed into the model. This allows the Random Forest to process the data, but at the cost of losing explicit temporal structure.

The model is trained using:

- A fixed number of decision trees (e.g., 200 estimators)
- Bootstrapped samples of the training data
- Random subsets of features for each split

Once trained, the model generates predictions for the test dataset, which are then evaluated using classification metrics and visual tools such as confusion matrices.

Figure 6 MLP Code

```
def run_rf(data):
    for name, (df, Xtr, Xv, Xte, Ytr, Yv, Yte, _) in data.items():
        print(f"\nRF → {name}")

        rf = RandomForestClassifier(n_estimators=200)

        rf.fit(
            Xtr.reshape(Xtr.shape[0], -1),
            Ytr
        )
        importances = rf.feature_importances_

        print("\nFeature Importance:")
        for f, imp in zip(FEATURES, importances[:len(FEATURES)]):
            print(f"{f}: {imp:.4f}")

        preds = rf.predict(
```

3.9. Training Strategy

Training deep learning models on financial time series data poses several challenges such as noisy data, class imbalances and overfitting to name but a few. To address some of these challenges different strategies were employed to make the efficient and have good results.

3.9.1. Focal loss to handle class imbalance

Financial Time Series Data such as the one on Financial Market is usually dominated or bias to the HOLD labels, with relatively fewer BUY and SELL labels signals. Standard loss functions tend to be swayed or biased to the hold signal because it is most of the different classes.

Focal loss addresses this challenge by reducing the importance of the dominant, easy classes (HOLD) and placed more significance on harder or misclassified classes. It helps the model to not only focus on hold but identify trading opportunities which make it easier to classify SELL and BUY.

This is very important because the ability of a trading strategy to be profitable relies mainly on accurately identifying trades rather than just predicting inactivity.

Figure 7 Class Imbalance

```
import tensorflow.keras.backend as K

def focal_loss(gamma=2., alpha=0.25):
    def loss(y_true, y_pred):
        y_pred = K.clip(y_pred, 1e-9, 1 - 1e-9)
        cross_entropy = -y_true * K.log(y_pred)
        weight = alpha * K.pow(1 - y_pred, gamma)
        return K.sum(weight * cross_entropy, axis=1)
    return loss
```

3.9.2. Class weights to emphasize minority classes

Class weights, in addition to focal loss, is applied on training the datasets. These performs its role by penalizing misclassification of minority classes. The combination of focal loss and class weights provides strong incentives for the model to adequately make meaningful decision between BUY, SELL and HOLD. It also helps to improve other metrics such as precision and f1 as accuracy alone in financial time series can be misleading.

However, excessive weighting often leads to instability in the model training, where the model tends to over predict minority classes. Therefore, weights are carefully tuned to balance sensitivity and precision

3.9.3. Early stopping to prevent overfitting

Early Stopping is a regularization mechanism in machine learning that halts training when a model's performance stops improving even if training accuracy keeps improving. Financial Time series data is inherently noisy, and continuous training may lead to overfitting.

Figure 8 Overfitting

```
early_stop = tf.keras.callbacks.EarlyStopping(  
    monitor='val_loss', #monitors validation loss  
    patience=10, # waits 10 epochs before stopping  
    restore_best_weights=True #keeps best performer  
)
```

Early stopping mitigates this by monitoring validation performance and halting training when improvement stalls. This ensures that the model retains its best-performing state rather than overfitting the training data.

3.10. Evaluation Framework

Traditional machine learning metrics such as accuracy and f1 score are not always reliable when it comes to trading models. A can still fail or not been profitable whilst having high accuracy if it fails to identify meaningful trades.

3.10.1. Backtesting

In trading backtesting is the process of testing a trading strategy against historical data to evaluate its performance, profitability and risk before embarking on that trading strategy with real capital. In machine learning backtesting converts model predictions into trading actions and applies them to historical data. This allows for the performance of the strategy to be evaluated based on financial outcomes rather than classification

Important considerations of the backtesting are that it aligns predictions and future returns, it also includes transaction cost and filters insignificant price movements. This is important because it ensures results are realistic and not fictional

3.10.2. Metrics

- **Sharpe ratio**

It was developed by Nobel laureate William F. Sharpe. The Sharpe ratio measures the return of the strategy relative to its risk. It is widely used in finance as a benchmark for risk-adjusted performance. It tells you how much excess return you are receiving for the extra volatility you endure for holding a riskier asset rather than a risk-free asset.

Formula 13- Sharpe Ratio

$$\text{Sharpe Ratio} = \frac{R_p - R_f}{\sigma_p}$$

Where:

R_p : Return of the portfolio or asset.

R_f : Risk-free rate (e.g., U.S. Treasury Bond yield).

σ_p : Standard deviation of the portfolio's excess return

A high Sharpe ratio indicates that the strategy generates consistent returns relative to its volatility.

- **Max drawdown**

Maximum drawdown measures the largest loss experienced by the strategy from a peak to a trough.

This is particularly important in trading, as investors are often more sensitive to losses than gains.

A strategy with high returns but large drawdowns may be considered too risky.

Formula 14 – Maximum Drawdown

$$\text{Maximum Drawdown (MDD)} = \frac{\text{Trough Value} - \text{Peak Value}}{\text{Peak Value}}$$

- **Win rate**

Win rate represents the proportion of profitable trades. While useful, it must be interpreted alongside other metrics, as a high win rate does not necessarily imply profitability.

For example, a strategy could win frequently but incur large losses occasionally.

Formula 15 – Win Rate

$$\text{Win Rate} = \left(\frac{\text{Number of Won Deals}}{\text{Total Number of Closed Opportunities}} \right) \times 100$$

3.11. Statistical Testing

Statistical tests are applied to returns generated by any strategy to ensure that results are not left to random chance. T-Test is used to analyse and determine the average returns is significantly different from zero. This is done to validate the reliability of the model.

3.12. Walk-Forward Validation

A major challenge of the traditional train/test split is that it does not represent the conditions of the financial market. This is because training done on historic data may not reflect future market conditions as the financial market keeps evolving.

To combat this challenge A walk-forward validation has been implemented. It iterates training on past window of data and testing on the immediate future, then shifts the window forward. This strategy prevents data leakage and preserves temporal order. This is done through the process of dividing the dataset into sequential segments for each iteration, and the model is trained on historical window and then tested on the immediately following period, the window is then moved forward and the cycle repeats. Through this it evaluates performance through different periods

Figure 9 Walk forward

```
def walk_forward_proper(model_builder, df, name, step=2000):  
    print(f"\nWALK-FORWARD → {name}")  
  
    scaler = StandardScaler()  
    all_sharpes = []  
  
    for start in range(0, len(df) - step*2, step*2): #walkforward loop  
  
        train_df = df.iloc[start:start+step]  
        test_df = df.iloc[start+step:start+step*2]  
  
        if len(test_df) < step:  
            continue  
  
        # FIT ONLY ON TRAIN  
        scaler.fit(train_df[FEATURES])  
  
        X_train_raw = scaler.transform(train_df[FEATURES])
```

4. Implementation

Advance and Modern Machine Learning models were used in the implementation of this trading system combined with data processing libraries, historical data of different financial market trading instruments, parameters, features and metrics chosen for the efficiency, compatibility of this type of time series data.

For this system to be successful it leverages the following core technologies:

4.1. Libraries

4.1.1. TensorFlow / Keras

TensorFlow and Keras are machine learning libraries that have an intrinsic relationship and complement each other, however they both help in the development of models both in the front and backend. For this system both TensorFlow and Keras are used in designing, training, developing and evaluating the deep learning models used such as CNN, LSTM etc. They were responsible for the preprocessing of the data which has to do with structuring the data and making it ready for the models. They also build and train models for it to be able to make its predictions

4.1.2. Scikit-learn for preprocessing

Scikit-learn is one of the most used python libraries because of its versatile use, ranging from preprocessing to classification and regression tasks. For this system Scikit learn as a library helps to main preprocessing tasks such as scaling features and maintaining class imbalance. Its robust and effective use helps to prevent data leakage and bias and prepare the data in the structure it is needed through processes like standardization and class weight computation. It also helps in the computation of metrics.

4.1.3. Pandas/NumPy for data handling

For this system the Pandas and NumPy libraries form the backbone of these systems because their uses and functionality cannot be over-emphasized when it comes to data exploration, manipulation and numerical computation. All the structured data handling, time indexing and feature engineering in this pipeline were performed by the Pandas while the NumPy concerns itself with efficient array operations required for model input preparations and mathematical computation. The datasets used in this pipeline are large with makes it apt that NumPy is used whilst Pandas handle any missing data in the dataset.

4.1.4. Matplotlib and Seaborn for visualization

For better understanding of the pipeline through analysis, interpretation and results the pipeline makes uses of visualization libraries which are matplotlib, and seaborn. These libraries are widely

used throughout the pipeline to better understand data exploration and model evaluation. They help in visualizing the following

- To exhibit portfolio growth overtime through Equity curves
- Training history plots, including loss and accuracy trends across epochs
- Confusion matrices, providing insight into classification performance across BUY, HOLD, and SELL classes
- Distribution plots, such as prediction confidence histograms
- Heatmaps, which improve interpretability of model errors and classification behaviour

Rather than relying solely on numerical metrics, visual tools allow for a more comprehensive evaluation of both predictive accuracy and trading effectiveness.

Figure 10 Libraries

```
import tensorflow as tf
tf.keras.backend.clear_session() # Clears old models from memory
import gc # Clears old models from memory (important in loops)

import numpy as np #numerical operations
import pandas as pd #handles data
import matplotlib.pyplot as plt #plots
import seaborn as sns
import mplfinance as mpf #handles candestick chart

from tensorflow.keras import layers
from tensorflow.keras.utils import to_categorical

from sklearn.preprocessing import StandardScaler #normalise data
from sklearn.utils.class_weight import compute_class_weight #handles class weight
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score #provides report

from sklearn.ensemble import RandomForestClassifier #rfm architecture
from imblearn.over_sampling import RandomOverSampler #duplicates minority classes to balance data
```

4.2. System Architecture Overview

The implementation of this system follows a strict modular pipeline, in which each stage of the pipeline is encapsulated in a dedicated function. This is done to improve readability, maintainability and of the system.

It is, however, divided into four parts, which each part performs a specific task. The tasks are hereby highlighted below:

4.2.1. Data Preparation – prepare_data()

The prepare_data function in the pipeline is the function that is responsible for preparing the data or preprocessing the data and it forms the backbone of the pipeline. It is responsible for turning raw financial time series data into structured data ready for machine learning. It performs a wide range of responsibilities highlighted below

- Loading and merging bid/ask datasets

- Performing feature engineering tasks
- Generating classification labels
- Splitting data into training, validation, and testing sets
- Applying feature scaling using training data only
- Converting tabular data into sequential inputs
- Handling class imbalance through weighting

Figure 11 Prepare Data

```
def prepare_data():
    datasets = {}

    for name, p in INSTRUMENTS.items(): #to loop through the different instruments
        print(f"\nPreparing {name}...")

        df = load_merge_bid_ask(p["ask"], p["bid"]) #this load and merge data
        df = add_features(df) #add the predefined features
        df = label_candles(df) #add the 3 labels

    #splits data into test, train and validation
    n1 = int(len(df) * TRAIN_RATIO)
    n2 = int(len(df) * (TRAIN_RATIO + VAL_RATIO))

    df_train = df.iloc[:n1].reset_index(drop=True)
```

This function in this stage helps to prevent data bias and leakage by fitting the scaler only on the training split data and applying and applying it to test and validation sets which makes it impossible for the pipeline to learn any information from the future.

Figure 12 Dataset

	time (eet)_x	AskOpen	AskHigh	AskLow	AskClose	volume_x \
0	2026.01.01 10:00:00	16.228	16.228	16.228	16.228	0.0
1	2026.01.01 10:05:00	16.228	16.228	16.228	16.228	0.0
2	2026.01.01 10:10:00	16.228	16.228	16.228	16.228	0.0
3	2026.01.01 10:15:00	16.228	16.228	16.228	16.228	0.0
4	2026.01.01 10:20:00	16.228	16.228	16.228	16.228	0.0

	datetime	time (eet)_y	BidOpen	BidHigh	BidLow \
0	2026-01-01 10:00:00	2026.01.01 10:00:00	16.197	16.197	16.197
1	2026-01-01 10:05:00	2026.01.01 10:05:00	16.197	16.197	16.197
2	2026-01-01 10:10:00	2026.01.01 10:10:00	16.197	16.197	16.197
3	2026-01-01 10:15:00	2026.01.01 10:15:00	16.197	16.197	16.197
4	2026-01-01 10:20:00	2026.01.01 10:20:00	16.197	16.197	16.197

	BidClose	volume_y	MidOpen	MidHigh	MidLow	MidClose	instrument
0	16.197	0.0	16.2125	16.2125	16.2125	16.2125	IUFSUSUSD_5m
1	16.197	0.0	16.2125	16.2125	16.2125	16.2125	IUFSUSUSD_5m
2	16.197	0.0	16.2125	16.2125	16.2125	16.2125	IUFSUSUSD_5m
3	16.197	0.0	16.2125	16.2125	16.2125	16.2125	IUFSUSUSD_5m
4	16.197	0.0	16.2125	16.2125	16.2125	16.2125	IUFSUSUSD_5m

instrument	
BCHUSD_10m	14661
IUFSUSUSD_5m	7168
XAUUSD_15m	6466
GBPUUSD_30m	3433

Name: count, dtype: int64

4.2.2. Model Training – run_cnn(), run_lstm(), run_mlp()

This function was responsible for the training and evaluation process of the different CNN, LSTM, MLP neural network architecture. The workflow for each model was consistent, and it took format as highlighted below.

- Model construction using predefined architecture
- Compilation with appropriate loss function and optimizer
- Training using historical sequences
- Validation monitoring with early stopping
- Prediction generation on unseen test data

Despite sharing a common structure, each model captures different aspects of the data:

- CNN focuses on short-term pattern recognition
- LSTM captures sequential dependencies over time
- MLP serves as a baseline without temporal awareness

Figure 13 CNN Model

```
def run_cnn(data):  
    for name, (df, Xtr, Xv, Xte, Ytr, Yv, Yte, class_weight_dict) in data.items(): #unpack prepare function  
        print(f"\nCNN → {name}")  
        model=build_cnn((SEQ_LEN, len(FEATURES))) #build model  
  
        #compile model  
        model.compile(optimizer="adam", loss=focal_loss(), metrics=["accuracy"])  
  
        #training  
        history = model.fit(  
            Xtr,  
            to_categorical(Ytr, NUM_CLASSES),  
            validation_data=(Xv, to_categorical(Yv, NUM_CLASSES)),  
            epochs=EPOCHS,  
            batch_size=BATCH_SIZE,  
            callbacks=[early_stop],  
            class_weight=class_weight_dict, # 🔥 ADD THIS  
            verbose=0  
        )
```

4.2.3. Backtesting Engine – run_backtest()

The model predictions made is converted by the backtesting component into simulated trading actions. This is a very important step as it closes the gap between predictions and real-world financial viability.

Some of the few functions the backtest function performs is

- The future market returns is align with predictions
- Trading logic is applied (BUY/SELL/HOLD)
- Transaction cost has been included
- The portfolio value has been updated over time

- Individual trades outcome has been recorded

Figure 14 Backtest

```
def run_backtest(preds, df, initial_balance=10000, cost=0.0002): #prediction, df, initial balance, cost
    returns = df["MidClose"].pct_change().shift(-1).values #compute price change, and aligns future return prediction

    balance = initial_balance #initial portfolio
    equity_curve = [balance]
    trades = []

    for i, p in enumerate(preds): #loop through predictions
        idx = i + SEQ_LEN + 6 #aligns predicition with actual price

        if idx >= len(returns): #skip if you do not have future data
            continue

        min_move = 0.001 # 0.1% minimum move filter

        if p == 1 and abs(returns[idx]) > min_move: #buy if price goes up
```

A key highlight of this implementation is how realistic constraints is been included such as trading cost and minimum price movement threshold. The stops the model from exploiting price fluctuations that will not be meaningful in real world. The output of this backtest function is an equity curve which shows portfolio growth and a list of trade returns.

4.2.4. Performance Evaluation – compute_metrics()

This function evaluates the trading strategy's performance and effectiveness using traditional machine learning metrics and financial performance metrics. These metrics are highlighted below It calculates:

- Risk-adjusted return (Sharpe ratio)
- Downside risk (maximum drawdown)
- Trade success rate (win rate)

This function was designed with the knowledge that classification accuracy alone is not enough to guarantee a successful trading system. Evaluation on a model must be done based on its ability to generate consistent and sustainable returns.

Figure 15 Metrics

```
def compute_metrics(equity, trades): #use equity and trades to calculate metrics
    returns = np.diff(equity) / equity[:-1] #converts equity cureve into percentage

    sharpe = 0
    if np.std(returns) > 0:
        sharpe = np.mean(returns) / np.std(returns) * np.sqrt(252*24)

    # max drawdown
    peak = np.maximum.accumulate(equity)
    dd = (equity - peak) / peak
    max_dd = dd.min()

    win_rate = 0
    if len(trades) > 0:
        win_rate = np.mean(trades > 0)

    return sharpe, max_dd, win_rate
```

4.2.5. Validation Framework – walk_forward_proper()

The pipeline includes walk-forward validation that simulates real-world deployment in order to ensure robustness. This function

- Train and test windows by splitting data sequentially
- The model for each window is then retraining
- Subsequent unseen period's performance is then evaluated
- Results across all windows are then aggregated.

Figure 15 Walkforward Function

```
def walk_forward_proper(model_builder, df, name, step=2000):
    print(f"\nWALK-FORWARD → {name}")

    scaler = StandardScaler()
    all_sharpes = []

    for start in range(0, len(df) - step*2, step*2): #walkforward loop

        train_df = df.iloc[start:start+step]
        test_df = df.iloc[start+step:start+step*2]

        if len(test_df) < step:
            continue

        # FIT ONLY ON TRAIN
        scaler.fit(train_df[FEATURES])

        | X_train_raw = scaler.transform(train_df[FEATURES])
          X_test_raw = scaler.transform(test_df[FEATURES])
```

This approach captures how the model adapts to changing market conditions unlike unmoved evaluation. It gives a more realistic assessment of performance and makes potential weaknesses in generalization.

5. Testing and Evaluation

The section of this research emphasis on the methods and strategies used in evaluating and testing the proposed trading pipeline. This was done to check the systems robustness, accuracy, practical applicability and performance. The aim and objective of this section is not only to prove the system is accurate but to also analyse its profitability tendencies. And in doing so different machine learning and trading strategies and metrics were used and this is explained in detail in the following paragraphs.

5.1. Model Performance

The Models performance is analysed and evaluated using two key areas which are the prediction accuracy and financial returns

5.1.1. Prediction accuracy

For this pipeline prediction accuracy typical mean how well the models can be able to classify the different market operations into BUY/SELL/HOLD categories. These are the 3 classes which can determine profitability. Since financial time-series data have high class imbalance, accuracy is not enough to determine the success of a particular model.

5.1.2. Financial returns

Whilst financial return, is the essence of any trading system, the models are evaluated based on real world trading scenarios and metrics such as Sharpe ratio, maximum drawdown and win rate. This helps identify the number of trades taken, and the profit probability of the model and from the total number of trades taken how many of them was won. This ensures how well the model performs financially and how it will perform in real world instead of just using statistical analysis.

5.2. Benchmark Comparison

To be able to determine whether the model is accurate and performing well. It has been compared as against two baseline strategies which is the buy and hold and the moving average strategy. These comparisons are very vital to show whether the machine learning models offer an advantage over traditional ones.

5.2.1. Buy & Hold

The Buy and Hold strategy are the major activity of the financial market. And for proper evaluation of the model's performance, it must outperform the baseline buy and hold, because it represents passive investment.

Figure 16 Buy & Hold Chart



5.2.2. Moving Average strategy

The moving average is a very vital and importance benchmark comparison for these models because it serves as a rule based technical trading benchmark. It captures trend actions and

5.2.3. Transaction Cost Analysis

To ensure realistic market trading evaluation the transaction cost analysis is be done using different levels such as 0.0001, 0.0002 and 0.0005.

Transaction cost is not only used to evaluate and test model but also it is an integral part of any trading system as it represents, spreads, commissions, and slippage that occurs in real trading markets. And different costs are used to test the different, results depending on which transaction cost as in real trading platform, each platform possess its own cost. The main aim of this test is to analyse whether trading cost both high and low is a determinant of a model's performance, because a strong model should be profitable even though the transaction cost increases or decreases.

Figure 17 Transaction cost

```
Transaction Cost Sensitivity:  
Cost 0.0001: Sharpe 0.423  
Cost 0.0002: Sharpe -0.299  
Cost 0.0005: Sharpe -2.396
```

5.3. Statistical Testing

For this pipeline to test the model statistically, T-test was used which determines whether the results produced by each model is statistically different from zero. To ascertain whether Predictive power or random chance as the driving force is the rational of using t-test as a statistical evaluation mechanism. A good statistical return shows that the model has been able to learn patterns adequately rather than overfitting or noise.

5.4. Ablation Study

In most trading pipeline it is difficult for one to able to test or evaluate the impacts on of the features used on the model. For this this pipeline this is done by removing some features and seeing how the model performs. Some features are removed and the model is retrained. The following feature groups are analysed.

- Technical indicators
- Candle features
- Price features

Through the ablation study it is much easier to identify the contribution of each feature to the model and compare their performance against other features. These such the influential features and least performing features or inputs. The ablation study also provides hidden details of model dependencies on specific information which is very vital for interpretability and optimisation.

Figure 18 Ablation Study

```
def run_ablation(data):
    print("\n==== ABLATION STUDY =====") #defines functions and prints headers to information is easy to read

    feature_sets = {
        "ALL": FEATURES,

        "NO_CANDLE": [f for f in FEATURES if not f.startswith("is_")],

        "ONLY_TECH": [
            "MACD", "Signal", "Jaw", "Teeth", "Lips",
            "stoch_k14", "stoch_k21", "stoch_k28"
        ],

        "ONLY_PRICE": [
            "body", "range", "upper_wick", "lower_wick", "body_ratio", "direction"
        ]
    }

    for name, (df, _, _, _, _, _) in data.items():
        print(f"\n{name}")
```

5.5. Error Analysis

An in-depth error analysis is performed to expose model weaknesses across different market conditions. These errors are analysed against the following

- Bullish regimes
- Bearish regimes
- Neutral markets

The regime-based analysis is done to know the model misclassification in identifying different patterns such as if the model performs well in high trending markets or when the market is volatile. The ability to be able to understand this different error analysis is key to improving robustness and design strategies that adapt to changing market conditions.

Figure 19 Error Analysis

30/30 ————— 2s 22ms/step

Error Analysis:

Total Error Rate: 0.6120042872454448

Error in Bullish: 0.4877126654064272

Error in Bearish: 0.7747524752475248

Error in Neutral: 0

6. Discussion

The main rationale of this research was to design, develop and evaluate a machine learning based trading pipeline consisting of different models that can adequately predict short term(intraday) financial market movements and converting those predictions into profitable trades. This pipeline was tested across different financial markets instruments, including foreign exchange (GBPUSD), commodities (XAUUSD), cryptocurrencies (BCHUSD), and equity indices (S&P 500) and with each instruments its own timeframe. The evaluation of model performance, ability, robustness under real market conditions and its implication and findings is discussed below.

6.1. Interpretation of Model Performance

6.1.1. Overall Model Performance

The **LSTM model applied to the IUFSUSUSD_5m dataset** has the highest sharper Sharpe ratio of **1.8757**, which shows the best performance of the research. Due to this result is a clear example that LSTM was able to make profitable trading signals with favorable risk-adjustment. The success of the LSTM model was due its capability to identify long term dependencies which is present in some of the datasets

Figure 19 LSTM vs Baseline

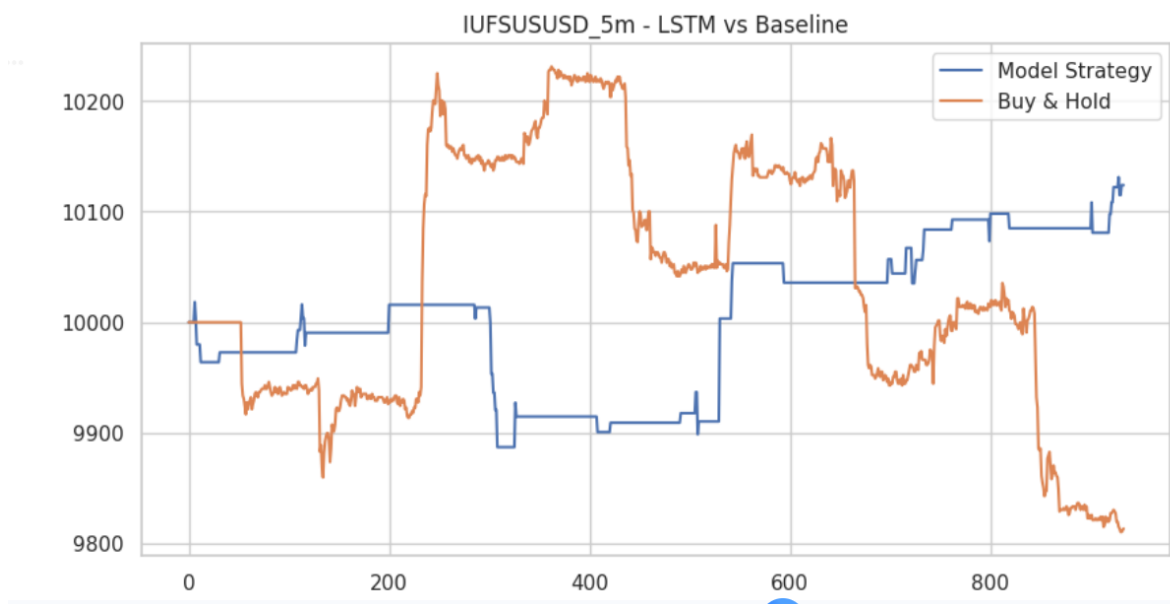


Figure 19 LSTM Classification Report

```
run_lstm(data)
```

Max DD: -1.31%

Win Rate: 7.31%

Sharpe Ratio: 1.876

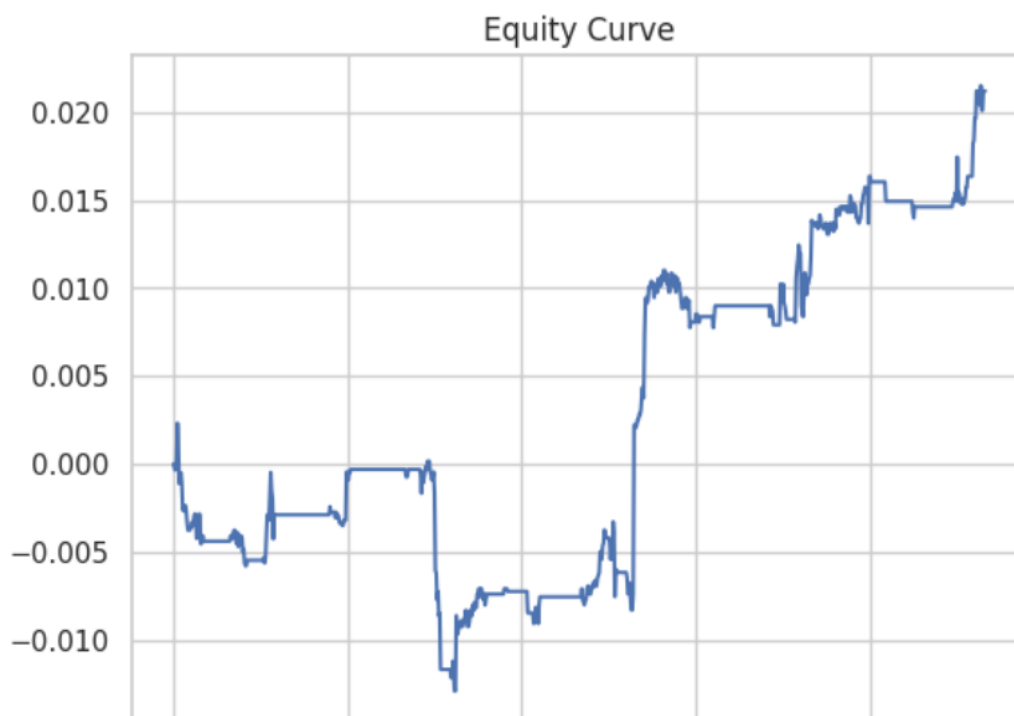
	precision	recall	f1-score	support
0	0.63	0.55	0.59	590
1	0.31	0.30	0.31	173
2	0.14	0.22	0.17	170
accuracy			0.44	933
macro avg	0.36	0.36	0.36	933
weighted avg	0.49	0.44	0.46	933

Total Errors: 521

False BUY: 115

False SELL: 220

Figure 20 LSTM Equity Curve



Second to the LSTM model in performance was the MLP Model on BCHUSD_10m with a Sharpe ratio of **1.6887**. This is a crucial outcome, as the MLP is a simple architecture compared to LSTM and CNN models. The success of the MLP shows that some markets such as cryptocurrency, you do not need complex modelling just better feature engineering can get you the result.

Figure 21 MLP vs Baseline



Figure 22 MLP Classification Return

Max DD: -0.86%

Win Rate: 38.71%

Sharpe Ratio: 1.689

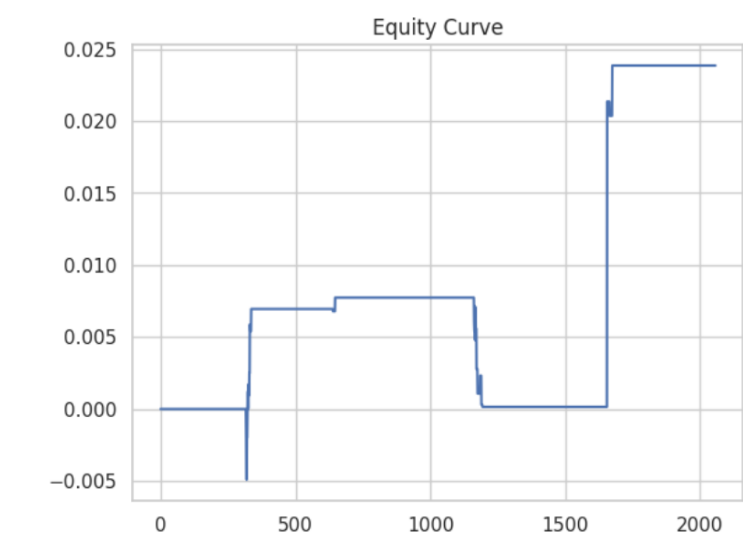
Total Errors: 1305

False BUY: 21

False SELL: 3

	precision	recall	f1-score	support
0	0.37	0.99	0.54	754
1	0.19	0.01	0.02	607
2	0.40	0.00	0.01	696
accuracy			0.37	2057
macro avg	0.32	0.33	0.19	2057
weighted avg	0.33	0.37	0.20	2057

Figure 23 MLP Equity Curve



6.1.2. Instrument-Specific Behaviour

All the models used has different performance on each instrument. And their performance depended on the financial instrument

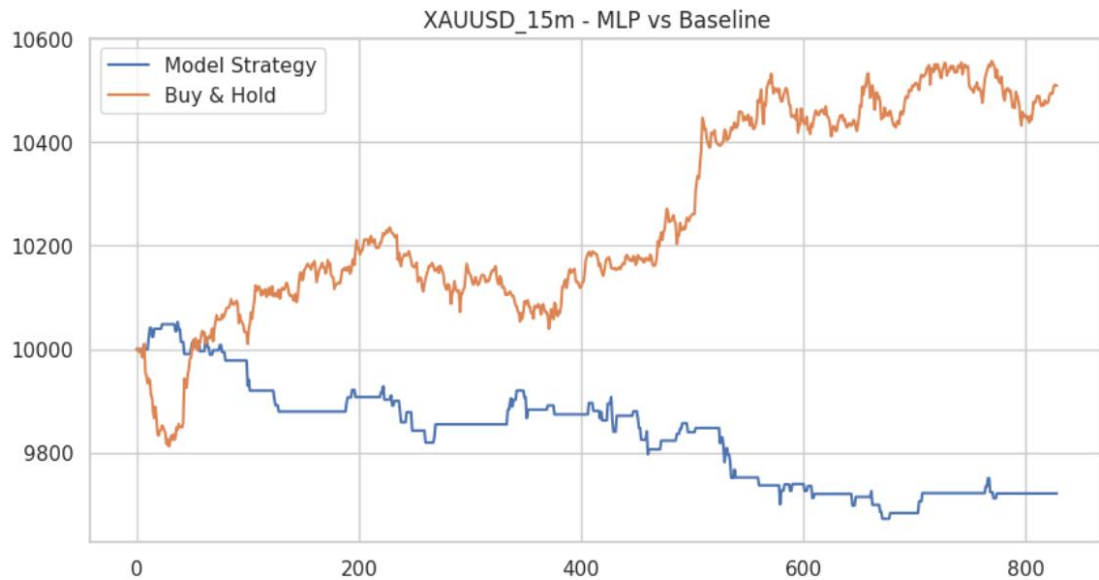
- According to the research IUFSUSUSD (**S&P 500**), the LSTM significantly overshadowed the other models, indicating the presence of usable temporal structure.
- For **BCHUSD (cryptocurrency)**, the MLP performed best, indicating that this market may possess more noise and less stable sequential patterns.
- For **GBPUSD (forex)**, instrument all models produced negative Sharpe ratios, with **Buy & Hold** performing better. This shows that the forex market in this dataset is highly efficient, making it difficult for predictive models to extract meaningful signals.

Figure 24 Classification Report for GBP/USD

...	Max DD: -0.14%				
	Win Rate: 0.90%				
	Sharpe Ratio: -0.154				
	Total Errors: 156				
	False BUY: 29				
	False SELL: 62				
		precision	recall	f1-score	support
	0	0.75	0.75	0.75	263
	1	0.26	0.15	0.19	68
	2	0.14	0.24	0.18	42
	accuracy			0.58	373
	macro avg	0.38	0.38	0.37	373
	weighted avg	0.59	0.58	0.58	373

- For **XAUUSD (gold)**, the **Buy & Hold** strategy achieved the highest Sharpe ratio (0.44), outperforming all machine learning models. This indicates that gold exhibited a strong underlying trend during the sample period, which was not effectively captured by the models.

Figure 25 Gold vs Baseline



These findings show that different markets exhibit different characteristics which means they require specific approach.

6.1.3. Performance of Individual Models

LSTM

The LSTM Model due to its ability to model sequential dependencies is appropriate for historical datasets where context is very vital. The LSTM shows strong performance in structured markets especially in IUFSUUSD. However, LSTM performance in IUFSUUSD was not replicate across other instruments, which shows that it is sensitive to noise, overfitting and less structured markets.

Table 1: LSTM Performance Table across different instruments

INSTRUMENT	SHARPE	ACCURACY	DRAWDOWN	WIN RATE
IUFSUUSD_5M	1.876	0.44	-0.01310	0.0731
BCHUSD_10M	-2.222	0.37	-0.16087	0.2988
XAUUSD_15M	-4.307	0.32	-0.04808	0.1621
GBPUSD_30M	-3.764	0.56	-0.00227	0.0090

MLP

From the overall performance across instruments the MLP model happens to be very robust despite its very simple architecture which aided it to having the second highest Sharpe ratio over other complex models. This is as a result that the feature engineering plays a much more vital role than

anticipated. In summary simpler models can perform well when provided with more information through inputs.

Table 2: MLP Performance Table across different instruments

INSTRUMENT	SHARPE	ACCURACY	DRAWDOWN	WIN RATE
IUFSUUSD_5M	-0.811	0.48	-0.02266	0.0553
BCHUSD_10M	1.689	0.37	-0.00859	0.3871
XAUUSD_15M	-3.637	0.33	-0.03785	0.1652
GBPUSD_30M	-0.154	0.58	-0.00144	0.0090

CNN

The CNN model consistently underperformed, with negative Sharpe ratios across most instruments. This may be since CNNs are designed to locate local spatial patterns, which may not convert effectively to financial time series data where patterns are less stable and more stochastic. Its best performance was on GBP/USD with a Sharpe Ratio of -0.299

Table 3: CNN Performance Table across different instruments

INSTRUMENT	SHARPE	ACCURACY	DRAWDOWN	WIN RATE
IUFSUUSD_5M	-4.149	0.39	-0.04699	0.0550
BCHUSD_10M	-2.885	0.36	-0.17691	0.2824
XAUUSD_15M	-2.193	0.34	-0.03392	0.1673
GBPUSD_30M	-0.299	0.49	-0.00171	0.0180

Figure 26: Classification Report

```

Transaction Cost Sensitivity:
... Cost 0.0001: Sharpe 0.423
Cost 0.0002: Sharpe -0.299
Cost 0.0005: Sharpe -2.396
Not statistically significant
MA Strategy Sharpe: -2.405
Max DD: -0.17%
Win Rate: 1.80%
Sharpe Ratio: -0.299

precision    recall  f1-score   support

0           0.72     0.56     0.63       263
1           0.21     0.24     0.22        68
2           0.19     0.40     0.26        42

accuracy          0.49       373
macro avg         0.37     0.40     0.37       373
weighted avg      0.57     0.49     0.51       373

Total Errors: 192
False BUY: 61
False SELL: 73

```

Ensemble

The ensemble model was the **weakest performer overall**, with consistently negative Sharpe ratios across all instruments and the worst result of **-7.243 on XAUUSD**. Rather than improving performance, the ensemble appeared to amplify errors from individual models. This suggests that the models being combined were not sufficiently diverse or individually strong, leading to correlated prediction errors.

Table 4: Ensemble Performance Table across different instruments

INSTRUMENT	SHARPE	ACCURACY	DRAWDOWN	WIN RATE
IUFSUUSD_5M	-3.445	0.43	-0.02632	0.0472
BCHUSD_10M	-2.584	0.35	-0.17849	0.3081
XAUUSD_15M	-7.243	0.29	-0.06950	0.1457
GBPUSD_30M	-1.069	0.48	-0.00171	0.0139

Figure 27 Classification Report

```
Error Analysis:
Total Error Rate: 0.7053140096618358
Max DD: -6.95%
Win Rate: 14.57%
Sharpe Ratio: -7.243

              precision    recall  f1-score   support

0               0.32         0.43         0.36         270
1               0.34         0.28         0.31         299
2               0.21         0.17         0.19         259

 accuracy                    0.29         828
 macro avg              0.29         0.29         0.29         828
 weighted avg           0.29         0.29         0.29         828

Total Errors: 584
False BUY: 165
False SELL: 167
```

6.2. Financial Performance vs Prediction Accuracy

A very important revelation of this research is the difference between classification accuracy and financial profitability. Models that possess moderate accuracy was still able to generate strong Sharpe Ratios which means high profitability tendencies. While in some cases the accuracies were higher but did not mean better trading performance (profitability).

This is explained by the asymmetric nature of financial returns. Adequately predicting large price movements is crucially valuable in trading than correctly predicting small or neutral movements.

Which means trading evaluation metrics must align with financial goals rather than just statistical assessments. Sharpe Ratio as the primary evaluation metrics used proves very vital, it allows the pipeline to prioritise consistency and risk -adjusted performance over just returns. This reiterates the importance of designing machine learning systems with domain specific evaluation criteria.

6.3. Benchmark Comparison

The Buy and Hold together with the Moving Average crossover strategy were the two strategies against which the models were benchmarked. According to results it was depicting that the Machine Learning models especially the LSTM and NLP approaches outperforms both benchmarks in terms of Sharpe Ratio.

The strategy that performed well especially in trending markets for assets such as equities and commodities is the Buy and Hold strategy though it suffered during the periods of volatility or sideways movement because it lacks adaptability. The machine learning models shows the ability to adjust to changing market conditions.

The Moving Average strategy was limited in it relying on lagging indicators but however provided a more active baseline. Though it captured trends it failed to react positively to short term fluctuations of the market. Since the machine learning model can capture more patterns beyond simple trends that leads to more superior performance.

6.4. Impact of Transaction Costs

Transaction cost analysis exposes crucial importance on strategy performance. As different transaction cost was used it was seen that the lower the transaction cost the increase in the Sharpe Ratio and the higher the cost the reduction in the Sharpe Ratio across all models. Though the degree of sensitivity varies between each model.

The LSTM and MLP models show higher adaptability to the transaction costs, maintaining good Sharpe Ratios even at higher cost levels. This indicates explains that these models generate stronger and reliable signals than the other models with sufficient edge to overcome trading frictions.

Weaker models such as CNN depicts that their signals were less robust and more sensitive to noise which leads to a decline in performance. This shows that the incorporation of transaction cost into evaluation cannot be overemphasized as ignoring can lead to misinterpretation and misdirected when in the real-world trading.

Figure 28: Transaction cost

```
Transaction Cost Sensitivity:  
Cost 0.0001: Sharpe 0.423  
Cost 0.0002: Sharpe -0.299  
Cost 0.0005: Sharpe -2.396  
Not statistically significant  
MA Strategy Sharpe: -2.405
```

6.5. Statistical Significance of Results

Apart from the fact that a metric in the financial domain was used to test and validate the models such as Shape Ratio, statistical testing was also used, and the t-test strategy was used to further validate the results of the model. The LSTM and MLP models show that it is significantly different from zero, showing that the performance is unlikely to be due to random chance.

However, the results of other instruments used in the pipeline was not statistically significant which shows weakness or less ability to predict. This depicts the existing difficulty of financial forecasting and the importance use of rigorous validation.

Statistical significance alone cannot guarantee economic significance, and it is very crucial that this should be always remembered. Even when returns are statistically significant, they must be adequately high and stable to be practically useful after taking into consideration costs and risk

6.6. Ablation Study and Feature Importance

The ablation study is very informative as it provides import insights into the role different features groups played in the research. It shows a noticeable decline in performance without technical indicators and price-based features. Which shows their importance in capturing market dynamics.

MACD and stochastic oscillators which are technical indicators contributed highly in detecting trend and momentum, while price structure features such as (body, wick etc..) captured market performance. Time based features especially cyclic encoding of hours and day was also important showing seasonality effects.

The irony of this research is that the candlestick pattern features had a limited impacted to the pipeline compared to other features used. Which shows that while patterns are very important in interpreting the value, they are less important in data driven modelling.

Overall, the ablation study shows that the use of diverse set of complementary features is essential for a very thorough pipeline.

Figure 29: Ablation study Result

```
run_ablation(data)
```

```
===== ABLATION STUDY =====
```

```
IUFSUSUSD_5m
44/44 ██████████ 2s 26ms/step
ALL: 0.5831
44/44 ██████████ 1s 13ms/step
NO_CANDLE: 0.5831
44/44 ██████████ 1s 13ms/step
ONLY_TECH: 0.5831
44/44 ██████████ 1s 13ms/step
ONLY_PRICE: 0.5831

BCHUSD_10m
91/91 ██████████ 2s 12ms/step
ALL: 0.3337
91/91 ██████████ 1s 7ms/step
NO_CANDLE: 0.3337
91/91 ██████████ 1s 6ms/step
ONLY_TECH: 0.3202
91/91 ██████████ 1s 6ms/step
```

6.7. Error Analysis Across Market Regimes

Error analysis shows that models performed differently in various market regimes, wherein in bullish conditions model were more accurate in predicting upward trend than downward likewise in bearish conditions they were more accurate in predicting downward trends. As a result, the highest error rates were seen in sideways markets or neutral markets. Which is not a surprise as the nature of financial time series data, where lack of decision or low volatility makes it very difficult for prediction to be made.

The use of the regime filter in the smart prediction function was effective in reducing errors. This was done by preventing the undertaking of trades that contradicts the existing trend. Thus, both accuracy and financial performance were improved. Shows the importance of machine learning and domain knowledge.

6.8. Walk-Forward Validation and Generalisation

Walk-Forward validation shows a realistic evaluation of model generalization. Unlike the usual test/train split this strategy mimic real-world scenarios retaining and testing the model of new data continuously.

Due to the changing market conditions, the results depict differences in Sharpe ratio across different windows. However, for the LSTM and MLP models the overall performance was positive. This shows how adaptive the models are to changing market conditions.

The differences noticed also shows the importance of robustness. For a trading system to be successful it must be consistent across different market environments rather than in just good conditions.

7. Conclusion

The aim of this research was to find out whether machine learning and deep learning methods can be applied to financial time series prediction and can be confidently transform into profitable trading strategies. This research went through rigorous steps in ensuring its profitability ranging from data collection, feature engineering, model training, backtesting and walk forward validation. This study shows a detail evaluation of predictive modelling in a financial context.

7.1. Limitations of the Study

Though some of the results are gotten from the models are very promising, it still has some limitations. The first of it is that the model was build using historical data which means some of these models might not perform well in future markets if these historic data do not correspond with future markets due to the dynamic nature of the financial markets.

Though a lot of features importance were used in other to make the model as complex and robust as possible it is possible that other relevant information might have been left out such macroeconomic indicators. Incorporating additional features or information will be able to massively improve the current pipeline.

Another limitation for this pipeline is that though the models were trained using limited horizons (6 timesteps) which directly aligns with intraday trading strategies, it might not perform well in capturing long term trends.

The backtesting strategy though realistic as it could be, does not account for all real-world conditions such as slippage, liquidity constraints and execution delays. Some of these factors highlighted can deeply hampered trading performance.

7.2. Implications and Future Work

What was discovered in this study has a lot of crucial implications. The first of which is that they display the effectiveness of deep learning models especially LSTM in financial time series prediction over traditional machine learning models. Secondly, they show the importance of aligning model with the type of data and the domain knowledge features.

There are various angles where future works can be explored first of which is the area of using more advance architectures such as attention-based models or transformers which could be better in sequence modelling.

Second of which is that reinforced learning could be implemented to optimise trading decisions rather than just classification. Additionally, a good direction where this model can be improved is by integrating alternative data sources such as news sentiment or macroeconomic indicators to improve predictive power. And lastly this model can be improved by incorporating a take profit/loss strategy which is a real-world trading situation that prove the robustness of the system.

While challenges remain, particularly in terms of generalisation and real-world implementation, the results indicate that data-driven approaches have significant potential to outperform traditional trading strategies. The project therefore contributes both practically and academically to the growing field of quantitative finance, providing a solid foundation for further research and development in intelligent trading systems

References

- Ahmed, N. K., Atiya, A. F., Gayar, N. E., et al. (2010). An empirical comparison of machine learning models for time series forecasting. *Econometric Reviews*, 2010, 594–621.
- Attigeri GV, MM MP, Pai RM, Nayak A (2015) Stock market prediction: a big data approach. In: IEEE region 10 conference on TENCON, pp 1–5
- Alostad H, Davulcu H (2015) Directional prediction of stock prices using breaking news on Twitter. In: IEEE/WIC/ACM international conference on WI-IAT 1
- Beck Thorsten, Ross Levine: Stock markets, banks, and growth
- Bi Bicong, Zhang Pinyi. Prediction of financial time series based on ARIMA-LSTM model [J]. statistics and decision, 2022,38(11):145-149.
- Box, G.E.P.; Jenkins, G.M.; Reinsel, G.C.; Ljung, G.M. *Time Series Analysis: Forecasting and Control*; John Wiley & Sons: Hoboken, NJ, USA, 2015; pp. 88–126. ISBN 978-111-867-4925.
- Chakraborty P, Pria US, Rony M, Majumdar MA (2017) Predicting stock movement using sentiment analysis of Twitter feed. In: IEEE 6th international conference ICIEV-ISCMHT, pp 1–6
- Chen W, Zhang Y, Yeo CK, Lau CT, Lee BS (2017b) Stock market prediction using neural network through news on online social networks. In: IEEE international ISC2, pp 1–6
- Chen L, Qiao Z, Wang M, Wang C, Du R, Stanley HE (2018) Which artificial intelligence algorithm better predicts the Chinese stock market? IEEE Access 6:48625–48633
- Cheng S, Shi Y, Qin Q, Bai R (2013) Swarm intelligence in big data analytics. In: International conference on intelligent data engineering and automated learning. Springer, Berlin, pp 417–426
- Dang LM, Sadeghi-Niaraki A, Huynh HD, Min K, Moon H (2018) Deep learning approach for short-term stock trends prediction based on two-stream gated recurrent unit network. IEEE Access 6:55392–55404
- Guresen, E., Kayakutlu, G., & Daim, T. U. (2011). Using artificial neural network models in stock market index prediction. *Expert Systems with Applications*, 38, 10389–10397.
- Heaton, J. B, Polson, N. G., & Witte, J. H. (2016). Deep learning in finance. arXiv preprint [arXiv:1602.06561](https://arxiv.org/abs/1602.06561).
- Dorffner, G.: Neural networks for time series processing, *Neural Network World* 4/96 (1996)
- Gershenfeld, N. A. and Weigend, A. S.: The future of time series, in: A. N. Gershenfeld and A. S. Weigen (eds), *Time Series Prediction: Forecasting the Future and Understanding the Past*, 1993
- Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. Cambridge: MIT Press.
- Hastie T, Tibshirani R, Friedman J (2009) The elements of statistical learning: data mining, inference, and prediction, 2nd edn. Springer, New York, p 745
- He K, Zhang X, Ren S, Sun J (2016) Deep residual learning for image recognition. In: Proceedings of IEEE conference on CVPR 2016, pp 770–778
- Hegazy O, Soliman OS, Salam MA (2014) A machine learning model for stock market prediction. Int J Comput Sci Telecommun 4(12):16–23
- Hsieh, T. J., Hsiao, H. F., & Yeh, W. C. (2011). Forecasting stock markets using wavelet transforms and recurrent neural networks: An integrated system based on artificial bee colony algorithm. *Applied soft computing*, 11, 2510–2525.

- Khan W, Malik U, Ghazanfar MA, Azam MA, Alyoubi KH, Alfakeeh AS (2019) Predicting stock market trends using machine learning algorithms via public sentiment and political situation analysis. *Soft Comput.*
- Khare K, Darekar O, Gupta P, Attar VZ (2017) Short term stock price prediction using deep learning. In: 2nd IEEE international conference RTEICT
- Kamijo, K.; Tanigawa, T. Stock price pattern recognition-a recurrent neural network approach. In Proceedings of the 1990 IJCNN International Joint Conference on Neural Networks, San Diego, CA, USA, 17–21 June 1990; Volume 1, pp. 215–221. Available online: <https://ieeexplore.ieee.org/abstract/document/5726532> (accessed on 18 March 2026).
- Krollner, B.; Vanstone, B.; Finnie, G. Financial time series forecasting with machine learning techniques: A survey. *Comput. Intell.* **2010**, *8*, 25–30
- Långkvist, M., Karlsson, L., & Loutfi, A. (2014). A review of unsupervised feature learning and deep learning for time-series modeling. *Pattern Recognition Letters*, *42*, 11–24.
- Liu Zhibin. Research on financial time series prediction based on hypergraph neural network and its application [D]. Shanxi University of Finance and Economics, 2024.
- Liu Jiaxuan. Research on multivariate time series prediction model based on deep learning [D]. wuhan textile University, 2024.
- Simonyan, K., & Zisserman, A. (2014). *Very deep convolutional networks for large-scale image recognition*. arXiv:1409.1556.
- Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence to sequence learning with neural networks. In *Advances in neural information processing systems* (pp. 3104–3112).
- Zhang Han, Wang Weiguo. Neural Network Integration Model for Financial Time Series Prediction [J]. Journal of Beijing University of Posts and Telecommunications, 2025,48(01):127-132.
- Zhang, G.; Qi, M. Neural network forecasting for seasonal and trend time series. *Eur. J. Oper. Res.* **2005**, *160*, 501–514